

基於預訓練P-Code 表示與圖域適應之少樣本跨架構IoT 惡意軟體分類

林禹丞¹, 鄭欣明²

台灣科技大學資訊工程系智慧型系統實驗室^{1, 2}
{m11215206、smcheng}@mail.ntust.edu.tw^{1,2}

摘要

物聯網 (IoT) 裝置廣泛運行於異質指令集架構 (ISA)，使得現有惡意軟體分析方法難以跨平台通用。本文提出一種以Ghidra High P-Code 為統一中介表示之跨架構IoT 惡意軟體偵測框架，透過結合預訓練語言模型 (Pre-trained Language Model, PLM) 與圖注意力網路 (Graph Attention Network, GAT)，同時捕捉P-Code 指令序列之語意特徵與函式呼叫圖之結構特徵，並引入域適應 (Domain Adaptation) 機制，在目標架構僅有少量標記樣本的條件下縮小跨架構特徵分佈差距。實驗結果顯示，單架構場景下各架構F1-macro 均超過94%；跨架構場景下，各架構交叉訓練測試的結果，F1-macro 均超過0.88 以上，驗證本框架之有效性。

關鍵字：跨架構惡意軟體分析、Ghidra P-Code、預訓練語言模型、圖神經網路、域適應

1. 前言

物聯網 (IoT) 生態系統正快速擴張，全球連網裝置數量於2025 年已達211 億[25]，廣泛部署於智慧家庭、工業控制、醫療照護等場域。不同於桌機或行動裝置集中於少數指令集架構 (Instruction Set Architecture, ISA)，IoT 裝置橫跨x86、ARM、MIPS、PowerPC 等異質ISA，且普遍缺乏完善的安全防護，導致IoT 裝置遭受的攻擊事件從2018 年的3,270 萬次急升至2022 年的1.1229 億次[17]，資安威脅日益嚴峻[19]。

設計IoT 惡意軟體分析機制面臨兩項主要挑戰：第一，正因IoT 生態系統橫跨高度異質的ISA，惡意軟體作者得以針對同一家族同時編譯多架構版本，感染更廣泛的裝置；因此，分析方法必須具備跨架構通用性，而非僅針對單一ISA 設計[1]；第二，相同的惡意行為在不同ISA 下呈現截然不同的指令特徵，使得基於位元組序列 (Byte Sequence) 與操作碼 (Opcode) 這些特徵的方法，針對單一架構所訓練的模型無法直接遷移。更現實的困難在於，惡意軟體樣本高度集中於x86-64 與ARM-32 等少數主流架構，其他低資源架構幾乎無標記資料可用。基於上述挑戰，有效的跨架構惡意軟體分析需要與架構無關的特徵，如：可印字串 (Printable String)、中間表示 (Intermediate Representation, IR)、統計特徵。本研究針對跨

架構惡意軟體家族分析問題，以Ghidra High P-Code 作為特徵，以預訓練語言模型 (Pre-trained Language Model, PLM) 對P-Code 指令序列進行深度語意編碼、以圖注意力網路 (Graph Attention Network, GAT) 建模函式呼叫圖結構，並引入域適應 (Domain Adaptation) 機制，在目標架構僅有少量標記樣本的條件下縮小跨架構特徵分佈差距。

本研究之核心研究問題 (Research Question, RQ) 為：我們該如何設計一個與架構無關 (architecture-independent) 的惡意軟體分析框架，使其在異質CPU 架構 (heterogeneous CPU architectures) 中仍能保持高效能？

本文之主要貢獻如下：

- **架構無關特徵設計：**以High P-Code 之操作碼序列為語意輸入，捨棄含架構相依資訊之varnode 位址空間，並移除Thunk 與外部函式雜訊，提升跨架構IR 一致性；在此基礎上以x86-64 的資料從頭預訓練RoBERTa 捕捉P-Code 語意，並結合GAT 建模函式呼叫圖之拓撲結構。
- **少樣本域適應：**引入圖域適應機制，在目標架構僅有每類5 筆標記樣本的條件下，有效縮小跨架構特徵分佈差距，大幅提升遷移表現。
- **嚴格跨架構評估：**有別於現有研究[27, 3, 21]將多架構混合訓練的做法，本研究明確定義訓練僅來自單一來源架構、測試來自另一目標架構的嚴格場景，並在涵蓋ARM-32、Intel、MIPS、x86-64 之資料集上以10 組隨機種子驗證框架有效性。

2. 相關文獻

2.1 跨架構惡意軟體分析

在真正具備跨架構能力的特徵方面，可印字串因直接來自二進位檔案中的可讀字元序列，不依賴特定ISA 的指令集，因此與架構無關：Lee 等人與Cheng 等人[9, 4]均以可印字串作為IoT 惡意軟體分類特徵，驗證其在跨架構場景下的可遷移性。然而可印字串語意稀疏，且易受字串混淆攻擊影響。以平台無關IR 為基礎的方法是另一類架構無關的特徵：CFDVex[18] 以VEX IR 結合CFG 特徵進

行跨架構IoT 惡意軟體偵測；CIMALiR[20] 以IR 進行跨平台惡意軟體聚類；在Assembly Code 方面，Redmond 等人[22]最早提出以NLP 方法學習跨架構指令嵌入，Unimap[31] 與Wang 等人[32]進一步透過對比式學習遷移，在語意空間去拉近兩個架構的相似度，提升嵌入品質實現跨架構偵測。

然後目前有許多研究[27, 3, 21]雖標榜跨架構，但實驗設置為將多個架構資料混合後共同訓練與測試，本質上仍依賴Byte 、Opcode 序列等受架構限制之特徵，訓練時已見過各架構樣本，並非以單一來源架構訓練、另一目標架構測試的場景進行實驗。此類設置無法反映真實世界環境所面臨的挑戰，其跨架構的表現存在高估，誤導了之後的跨架構惡意軟體分析研究。

2.2 中間表示與Ghidra P-Code

利用平台無關的IR 是消除ISA 差異的主要策略之一。CFDVex[18] 以VEX IR 為基礎，結合CFG 特徵進行IoT 惡意軟體偵測；CIMALiR[20] 以IR 進行跨平台IoT 惡意軟體聚類；Greco 等人[7]以IR 為基礎，搭配YARA 語法產生平台無關之惡意軟體特徵碼，抵抗常見混淆技術。在二進位函式相似性分析方面，VulHawk[13]、VexIR2Vec[29] 與Li 等人[10]分別以IR 為核心，透過圖結構表示與嵌入學習實現跨架構函式相似性偵測。我們先前的工作[11]以Ghidra P-Code 統計特徵分類IoT 惡意軟體，發現單架構準確率高達99.9%，但跨架構場景下性能嚴重衰退，揭示純統計特徵之語意對齊局限。

Ghidra[15] 為NSA 開發之開源逆向工程工具，其P-Code[6] 是一種RTL 中間表示。P-Code 相關文獻與文檔目前仍相當有限：Naus 等人[16]為P-Code 建立形式語意，並在此過程中發現官方文件存在多處不完整與描述不精確的問題。P-Code 分為兩層：Low P-Code 由SLEIGH 將機器碼直接展開，保留架構特定暫存器細節；High P-Code 則由反編譯器進一步進行資料流分析（包含靜態單賦值（Static Single Assignment, SSA）轉換、死碼消除（Dead Code Elimination）、變數識別）後產生，消除架構特定細節。每條P-Code 指令由Varnode、Address Space 與P-Code Operation 三部分組成，共定義71 個操作碼，依功能分為資料搬移、控制流程、整數算術、整數比較、整數位元、浮點運算及布林與轉換等類別。如圖 1 所示，MIPS、ARM 與x86-64 的High P-Code 在同一函式下趨於一致；然而我們先前的工作[11]與Naus 等人[16]均指出，即便在編譯器（compiler）與優化等級（optimization level）相同的條件下，相同原始碼跨架構編譯所產生的二進制檔案的P-Code 指令序列仍存在差異；在真實世界的惡意軟體分析場景中，編譯環境更加多樣，差異只會更大，因此語意對齊仍需額外處理。本研究即以High P-Code Operation 作為特徵，後面文字會以P-Code 作為簡稱。

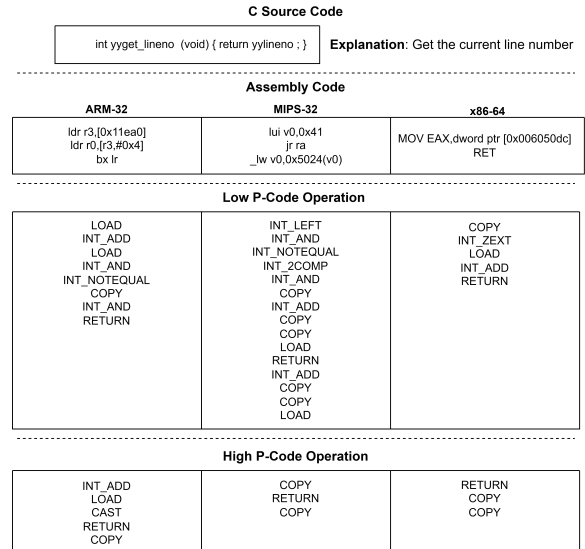


圖 1 同一C 函式在MIPS、ARM 與x86-64 架構下之比較

3. 方法

本節介紹所提方法之整體架構，包含特徵提取與前處理、預訓練語言模型、圖注意力網路與域適應四個主要模組。圖 3 為整體框架概覽，圖 2 為特徵抽取流程。

3.1 特徵提取與前處理

本研究以Ghidra[15] 對ELF 二進位檔進行靜態分析，提取函式呼叫圖（Function Call Graph, FCG）與High P-Code。為降低雜訊並提升跨架構一致性，過濾掉Thunk Function 與External Function，僅保留具實質語意的函式節點。各函式節點之P-Code 序列作為語意輸入，函式間呼叫關係構成FCG 的邊。

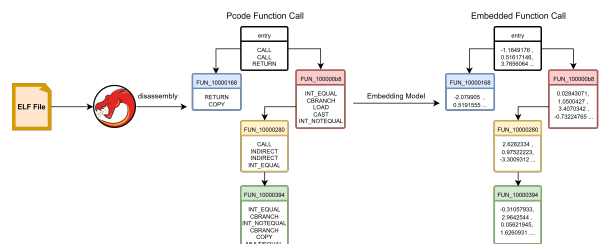


圖 2 特徵處理流程：從ELF 二進制檔案經Ghidra 轉換為函數呼叫圖，再進行圖嵌入。

3.2 預訓練語言模型

本研究採用RoBERTa [12] 對各函式節點的P-Code 序列進行語意編碼。由於P-Code 序列長度遠長於Opcode 等特徵，RoBERTa 的Transformer 架構能透過自注意力機制有效捕捉指令間的長程依

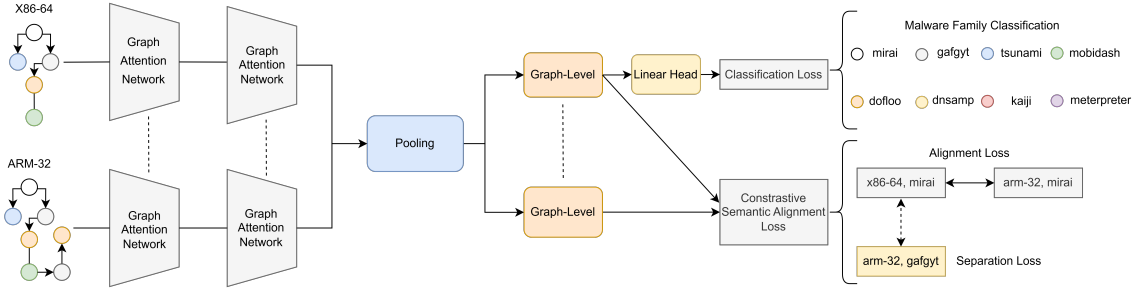


圖 3 所提跨架構惡意軟體偵測框架整體架構概覽。

賴，並以輕量架構從頭預訓練以適應P-Code 領域語彙（詳細超參數設定於第4.1 節）。

Tokenizer. 由於P-Code 操作碼來自有限的指令集，本研究採用詞層級分詞（Word-level Tokenization），以空白字元為切分邊界，掃描訓練語料中所有唯一的P-Code token 建立詞彙表，並加入[CLS]、[SEP]、[MASK]、[PAD]、[UNK] 五個特殊token。每條序列經tokenizer 後，自動在首尾插入[CLS] 與[SEP]，再截斷或填補至最大長度512。

給定函式節點 v 對應的P-Code token 序列 $\mathbf{x}_v = (t_1, t_2, \dots, t_L)$ ，RoBERTa 輸出各位置的隱藏狀態（hidden state） $\{\mathbf{h}_1, \dots, \mathbf{h}_L\}$ ，節點嵌入以mean pooling 計算：

$$\mathbf{e}_v = \frac{1}{L} \sum_{l=1}^L \mathbf{h}_l \quad (1)$$

如圖 4 所示，本研究以MLM 任務從頭預訓練。MLM 由BERT [5] 所提出，對每條輸入序列隨機選取15% 的token 位置構成遮蔽集合 $\mathcal{M}$ ；被選中的位置中，80% 替換為[MASK]，10% 替換為詞彙表中的隨機token，10% 保持原token 不變。訓練目標為最小化被遮蔽位置的交叉熵損失：

$$\mathcal{L}_{\text{MLM}} = - \sum_{i \in \mathcal{M}} \log P(t_i | \mathbf{x}_{\setminus \mathcal{M}}) \quad (2)$$

其中 $\mathcal{M}$ 為被遮蔽位置之集合。各架構下超過92% 的函式節點token 長度不超過512，序列上限設為512 可覆蓋絕大多數函式。

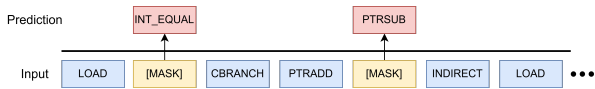


圖 4 以MLM 任務預訓練：隨機遮蔽序列中的P-Code後，訓練模型預測被遮蔽的token，使模型學習P-Code 序列的語意規律。

3.3 圖神經網路

本研究以FCG 作為圖結構輸入，每個節點代表一個函式，初始節點特徵由RoBERTa 所產生的嵌入 $\mathbf{e}_v$ 提供（即 $\mathbf{h}_v^{(0)} = \mathbf{e}_v$ ），函式間呼叫關係構成圖的邊集合 $\mathcal{E}$ 。本研究採用圖注意力網路（Graph

Attention Network, GAT）[28] 對節點表示進行聚合，透過注意力機制自適應地加權鄰居貢獻。模型共設置2 層：第一層使用4 個注意力頭（multi-head attention）並將結果拼接輸出；第二層則使用單頭注意力，輸出維度 $d = 256$ 的最終節點表示 $\mathbf{h}_v^{(K)}$ 。

注意力池化（Attentional Pooling） 為將圖中所有節點表示聚合為固定維度的圖級嵌入（graph-level embedding），本研究採用注意力池化機制。對圖 $\mathcal{G}$ 中每個節點 v ，以閥控網路（gate network） f_{gate} 計算其重要性分數：

$$e_v = f_{\text{gate}}(\mathbf{h}_v^{(K)}) = \mathbf{W}_2 \text{ReLU}(\mathbf{W}_1 \mathbf{h}_v^{(K)}) \quad (3)$$

其中 $\mathbf{W}_1 \in \mathbb{R}^{128 \times 256}$ 、 $\mathbf{W}_2 \in \mathbb{R}^{1 \times 128}$ ，接著以softmax 正規化後加權加總所有節點表示：

$$\mathbf{z} = \sum_{v \in \mathcal{G}} \frac{\exp(e_v)}{\sum_{u \in \mathcal{G}} \exp(e_u)} \mathbf{h}_v^{(K)} \quad (4)$$

得到圖級嵌入 $\mathbf{z} \in \mathbb{R}^{256}$ ，作為後續分類與對比對齊的輸入。

分類器 圖級嵌入 $\mathbf{z}$ 輸入線性分類器 $\mathbf{W}_c \in \mathbb{R}^{|\mathcal{Y}| \times 256}$ 進行家族分類，輸出各類別的logit，再以交叉熵損失 $\mathcal{L}_C$ 監督訓練。

3.4 域適應與對比語意對齊

跨架構泛化的核心挑戰在於協變量偏移（Covariate Shift）[24]。第2 節我們針對P-Code 這個特徵的觀察，即使函式是一樣的原始碼，編譯至不同的指令集架構後，其P-Code 序列與FCG 特徵仍會產生顯著差異，導致源架構與目標架構的特徵邊際分佈並不相同（即 $P(\mathbf{X}^s) \neq P(\mathbf{X}^t)$ ）[2]。本研究的直接遷移實驗（Lower Bound）進一步證實此偏移的影響：僅以源架構訓練的GAT 模型會對其特徵分佈過擬合，導致映射至潛在空間的圖級嵌入仍帶有強烈的架構偏差（ $P(\mathbf{Z}^s) \neq P(\mathbf{Z}^t)$ ），進而造成效能嚴重衰退。

為克服此特徵分佈差異，現有的無監督域適應（Unsupervised Domain Adaptation, UDA）方法通常透過對齊整體的機率分佈來進行特徵轉換[26]。然而，文獻[14] 指出，若能進一步利用目標域中既有的少量標記樣本，則可引導模型實現更精確的類別層級對齊（Semantic Alignment）。基於此概念，本研究採用監督式域適應策略，引

入並擴展Motiian 等人提出的分類與對比語意對齊 (CCSA) 框架 [14]，該框架最初在影像分類任務中取得成功，本研究將其應用至跨架構惡意軟體分析中，結合圖神經網路 (GAT) 提取圖級嵌入。此方法在目標架構僅有極少數標記的條件下，有效實現跨架構的語意對齊與特徵分離。

設 $\mathbf{z}_i^s$ 與 $\mathbf{z}_j^t$ 分別為源架構與目標架構樣本經 GAT 與注意力池化所得之圖級嵌入，於池化後施以 L2 正規化， y_i^s 與 y_j^t 為對應的家族標籤。定義正樣本對集合 $\mathcal{P} = \{(i, j) \mid y_i^s = y_j^t\}$ 與負樣本對集合 $\mathcal{N} = \{(i, j) \mid y_i^s \neq y_j^t\}$ [8]。

語意對齊損失 (Semantic Alignment Loss) 縮短同家族跨架構嵌入之距離：

$$\mathcal{L}_{SA} = \frac{1}{|\mathcal{P}|} \sum_{(i,j) \in \mathcal{P}} \|\mathbf{z}_i^s - \mathbf{z}_j^t\|^2 \quad (5)$$

類別分離損失 (Separation Loss) 對不同家族的跨架構嵌入施加 margin 懲罰：

$$\mathcal{L}_{Sep} = \frac{1}{|\mathcal{N}|} \sum_{(i,j) \in \mathcal{N}} \max(0, m - \|\mathbf{z}_i^s - \mathbf{z}_j^t\|)^2 \quad (6)$$

其中 m 為 margin 超參數，控制不同類別嵌入在空間中的最小分離程度。最終訓練目標結合分類損失 $\mathcal{L}_C$ (交叉熵) 與上述兩項對比對齊損失：

$$\mathcal{L}_{total} = (1 - \alpha) \mathcal{L}_C + \alpha (\mathcal{L}_{SA} + \mathcal{L}_{Sep}) \quad (7)$$

其中 $\alpha \in [0, 1]$ 平衡分類與對比對齊的比重。

4. 實驗

本節系統性地評估所提框架在單架構與跨架構場景下的性能。

4.1 實驗設置

實驗環境為配備 NVIDIA GeForce RTX 5070 Ti GPU 之工作站。

預訓練語言模型 (RoBERTa)：採用 6 層 Transformer 架構，隱藏維度 256，8 個注意力頭，前饋層維度 1024，最大序列長度 512。以約 600 萬筆 x86_64 架構函式節點之 P-Code 序列作為預訓練語料，以遮罩語言模型 (MLM) 任務從頭預訓練，遮罩比例 15%，Adam 優化器，學習率 $1e-4$ ，Warmup 10,000 步，線性排程器，批次大小 64，訓練 100 個 epoch。

GAT：採用 2 層架構，隱藏維度 256，Dropout 0.2，注意力池化 (Attention Pooling)。Adam 優化器，學習率 0.001，批次大小 32，最多訓練 200 個 epoch，Early Stopping (patience=15)，損失函式為 Cross Entropy。

CCSA：以上述 GAT 架構為骨幹，加入對比語意對齊損失。損失權重 $\alpha = 0.3$ ，margin $m = 0.5$ ，目標域每類取 5 筆標記樣本 (Few-shot)，訓練時正負樣本對比例為 1:3。

重複實驗：單架構實驗與直接遷移 (Lower Bound) 因訓練與測試樣本固定，僅執行一次；跨

架構實驗 (Few-shot RF Baseline 與 Our Method) 中，不同隨機種子會改變目標域 few-shot 訓練樣本、來源域訓練/驗證切割及測試樣本的組成，以 10 組不同隨機種子重複執行，並取 F1-macro 之平均值與標準差。

4.2 資料集

本研究收集涵蓋 ARM-32、ARM-64、Intel i386、MIPS、PowerPC (PPC) 及 AMD x86-64 共六種架構之 IoT 惡意軟體樣本。所有樣本透過 VirusTotal[30] 進行多引擎掃描，並以 AVClass[23] 自動化工具統一標記惡意軟體家族，最終涵蓋十個家族，原始資料集共 36,078 個樣本。

在前處理階段，我們首先移除加殼樣本，因打包技術會掩蓋惡意軟體的固有特徵並扭曲分類結果；其次移除重複樣本，以防止訓練與測試階段之間發生資料洩漏。此外，部分二進位檔因逆向工程失敗而被排除：逾時 (超過 600 秒) 245 筆、節區格式錯誤 1,432 筆、無邊 1,017 筆、空節點 72 筆，共計移除 2,766 筆。最終有效資料集共 33,312 個樣本，各家族與架構之詳細分布如表 1 所示。

4.3 實驗資料切割

為確保各實驗具備可比較性，本研究針對單架構與跨架構兩種場景分別制定資料切割策略。由於 IoT 真實環境所蒐集之樣本在各架構與各家族之間分布高度不均 (如表 1)，部分家族在特定架構下樣本數極少，直接隨機切割將導致測試集過小而評估不具代表性。為此，我們採用以下統一切割原則：測試集每個家族各取約 50 個樣本；訓練集每個家族至多保留 300 個樣本。此一原則同時適用於單架構、跨架構與跨架構遷移學習三種實驗設置，確保三者採用相同的評量資料，具備公平可比較的基準。依此原則，最終納入實驗之樣本涵蓋 ARM-32、Intel、MIPS、x86_64 共四種架構，以及 dnsamp、dofloo、gafgyt、kaiji、meterpreter、mirai、mobidash、tsunami 共八個惡意軟體家族。各家族在不同架構下之訓練樣本數介於 47 至 300 之間，樣本數充足之家族 (如 gafgyt、mirai 等) 各架構均達上限 300 筆，樣本較稀缺之家族 (如 dnsamp、meterpreter) 則取全部可用樣本。

4.4 單架構分類

單架構實驗目的在於建立各架構下的性能上界，確認模型在同架構訓練與測試時的分類能力。PLM (RoBERTa) 以 x86_64 架構資料進行預訓練；GAT 各自以當前架構之資料進行訓練與測試，採 60/20/20 之訓練、驗證、測試比例分割。各架構均可達 F1-macro 94% 以上，驗證所提框架在同架構場景下具備穩定之分類能力。

表 1 實驗資料集各惡意軟體家族與架構之樣本數分布

Family	ARM-32	ARM-64	Intel	MIPS	PPC	x86_64	Total
mirai	1,018	331	1,083	1,651	3,218	1,185	8,486
gafgyt	1,054	20	1,094	1,715	3,214	1,203	8,300
tsunami	983	25	1,095	1,683	530	1,206	5,522
mobidash	1,063	284	1,098	306	0	1,216	3,967
dofloo	1,062	0	1,095	365	0	129	2,651
meterpreter	137	46	598	47	24	1,154	2,006
kaiji	134	91	211	164	0	244	844
wroba	467	49	284	0	0	2	802
dnsamp	95	0	99	64	0	208	466
fakebank	217	0	51	0	0	0	268
Total	6,230	846	6,708	5,995	6,986	6,547	33,312

4.5 跨架構遷移學習比較

表 2 整合單架構與跨架構各設定之F1-macro，以統一評量指標進行比較，共四種設定：

Upper Bound (單架構)：Source 與Target 為同一架構，代表理想情境下的性能上界。

Lower Bound (跨架構Source Only)：GAT 以Source 架構全量資料訓練，直接在Target 架構測試，不進行任何適應，代表跨架構遷移之下界。

Few-shot Baseline (RF)：不使用Source 資料，僅從Target 架構每個家族取5 筆樣本，對GNN Embedding 做Mean Pooling 後訓練Random Forest (200 棵樹，max_features=sqrt)。我們比較了XGBoost、SVM 及Random Forest 多種模型與參數組合，Random Forest 表現最佳，故以此為代表。

Our Method (跨架構+ DA)：在Source 架構訓練基礎上，引入對比語意對齊域適應框架，利用Target 架構每個家族5 筆有標記樣本進行對齊微調。

表 2 單架構與跨架構各設定F1-macro 比較 (mean $\pm$ std)

Architecture	F1-macro	
	Upper Bound	Few-shot RF
ARM-32	0.9613 $\pm$ 0.0095	0.7978 $\pm$ 0.0339
Intel	0.9781 $\pm$ 0.0049	0.8256 $\pm$ 0.0365
MIPS	0.9448 $\pm$ 0.0042	0.8010 $\pm$ 0.0375
x86_64	0.9653 $\pm$ 0.0154	0.8163 $\pm$ 0.0314
Source $\rightarrow$ Target	Lower Bound	Our Method
x86_64 $\rightarrow$ ARM-32	0.4659	0.9249 $\pm$ 0.0156
x86_64 $\rightarrow$ Intel	0.8483	0.9178 $\pm$ 0.0040
x86_64 $\rightarrow$ MIPS	0.2228	0.8806 $\pm$ 0.0288
ARM-32 $\rightarrow$ Intel	0.7800	0.8916 $\pm$ 0.0147
ARM-32 $\rightarrow$ MIPS	0.5271	0.9214 $\pm$ 0.0295
ARM-32 $\rightarrow$ x86_64	0.6723	0.8939 $\pm$ 0.0061
Intel $\rightarrow$ ARM-32	0.6993	0.9323 $\pm$ 0.0104
Intel $\rightarrow$ MIPS	0.4398	0.8915 $\pm$ 0.0133
Intel $\rightarrow$ x86_64	0.9185	0.9491 $\pm$ 0.0076

Upper Bound 各架構均達F1 > 0.94，確立性能上界。Lower Bound 代表分佈偏移的結

果：x86_64 直接測試於MIPS 僅0.2228，ARM-32 僅0.4659。Few-shot Baseline 在不使用Source 資料的條件下，僅以每類5 筆樣本達F1 約0.80，可作為資料稀缺場景之基準。本研究之方法均達F1 $\geq$ 0.88，最高為Intel $\rightarrow$ x86_64 之0.9491，較各對應Lower Bound 平均提升0.29。

5. 結論

本文提出一種以Ghidra High P-Code 為統一中介表示之跨架構IoT 惡意軟體分析框架，結合RoBERTa PLM 與GAT，同時建模P-Code 指令序列之語意特徵與FCG 之結構特徵。針對目標架構標記資料匱乏的挑戰，本研究引入監督式域適應機制，僅需每類5 筆目標域樣本縮小跨架構特徵分佈差距。實驗結果顯示，單架構場景下各架構F1-macro 均超過94%；跨架構場景下均達0.88 以上，相較Lower Bound 平均提升0.29，最高提升0.66。未來工作將探索更有效的嵌入空間對齊策略，並進一步分析跨架構嵌入空間之結構與可解釋性。

參考文獻

- [1] M. Alhanahnah, Q. Lin, Q. Yan, N. Zhang, and Z. Chen. Efficient signature generation for classifying cross-architecture IoT malware. In *Proc. IEEE Conf. Communications and Network Security (CNS)*, pages 1–9, 2018.
- [2] J. Blitzer, R. McDonald, and F. Pereira. Domain adaptation with structural correspondence learning. In *Proceedings of the 2006 conference on empirical methods in natural language processing*, pages 120–128, 2006.
- [3] R. Chaganti, V. Ravi, and T. D. Pham. Deep learning based cross architecture internet of things malware detection and classification. *Computers & Security*, 120:102779, 2022.
- [4] Y.-S. Cheng, T. Ban, S.-M. Cheng, T. Takahashi, and W.-C. Teng. Lightweight, architecture-agnostic IoT malware detection

- via printable strings. In *Proc. 2025 Workshop on Recent Advances in Resilient and Trustworthy MACHine learning-driveN Systems*, pages 24–33, 2025.
- [5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186, 2019.
 - [6] Ghidra. P-Code reference manual. [Online]. Available: https://ghidra.re/ghidra_docs/languages/html/pcoderef.html, 2023. Accessed: Feb. 2026.
 - [7] C. Greco and M. Ianni. A cross-architecture malware detection approach based on intermediate representation. *Journal of Information Security and Applications*, 93:104117, 2025.
 - [8] R. Hadsell, S. Chopra, and Y. LeCun. Dimensionality reduction by learning an invariant mapping. In *2006 IEEE computer society conference on computer vision and pattern recognition (CVPR'06)*, volume 2, pages 1735–1742. IEEE, 2006.
 - [9] Y.-T. Lee et al. Cross platform IoT-malware family classification based on printable strings. In *Proc. IEEE 19th Int. Conf. Trust, Security and Privacy in Computing and Communications (TrustCom)*, pages 775–784, 2020.
 - [10] X. Li, J. Sun, Y. Zhang, X. Yin, and S. Liu. Binary similarity detection based on intermediate representation graph fusion. In *Proceedings of the 2025 4th International Conference on Cryptography, Network Security and Communication Technology*, pages 106–114, 2025.
 - [11] Y.-C. Lin et al. Cross-architecture IoT malware analysis using P-Code intermediate representation. In *Proc. Workshop on Recent Advances in Resilient and Trustworthy MACHine learning-driveN Systems (RTMAN)*, pages 45–50, 2025.
 - [12] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.
 - [13] Z. Luo et al. VulHawk: Cross-architecture vulnerability detection with entropy-based binary code search. In *Proc. NDSS*, 2023.
 - [14] S. Motiian, M. Piccirilli, D. A. Adjeroh, and G. Doretto. Unified deep supervised domain adaptation and generalization. In *Proceedings of the IEEE international conference on computer vision*, pages 5715–5725, 2017.
 - [15] National Security Agency. Ghidra. [Online]. Available: <https://github.com/NationalSecurityAgency/ghidra>. Accessed: Feb. 2026.
 - [16] N. Naus et al. A formal semantics for P-Code. In *Working Conf. Verified Software: Theories, Tools, and Experiments*, pages 111–128. Springer, 2022.
 - [17] A. Petrosyan. Distribution of cyber-attacks across worldwide industries in 2023. Statista. [Online]. Available: <https://www.statista.com/statistics/1315805/cyber-attacks-top-industries-worldwide/>, 2024. Accessed: Feb. 2026.
 - [18] T. N. Phu et al. CFDVex: A novel feature extraction method for detecting cross-architecture IoT malware. In *Proc. 10th Int. Symp. Information and Communication Technology*, pages 248–254, 2019.
 - [19] S. U. Qureshi, J. He, S. Tunio, N. Zhu, A. Nazir, A. Wajahat, F. Ullah, and A. Wadud. Systematic review of deep learning solutions for malware detection and forensic analysis in IoT. *Journal of King Saud University - Computer and Information Sciences*, page 102164, 2024.
 - [20] M. A. Rahat et al. CIMALiR: Cross-platform IoT malware clustering using intermediate representation. In *Proc. IEEE 14th Annual Computing and Communication Workshop and Conference (CCWC)*, pages 0460–0466, 2024.
 - [21] V. Ravi, T. D. Pham, and M. Alazab. Attention-based multidimensional deep learning approach for cross-architecture IoMT malware detection and classification in healthcare cyber-physical systems. *IEEE Transactions on Computational Social Systems*, 10(4):1597–1606, 2022.
 - [22] K. Redmond, L. Luo, and Q. Zeng. A cross-architecture instruction embedding model for natural language processing-inspired binary code analysis. *arXiv preprint arXiv:1812.09652*, 2018.
 - [23] M. Sebastián, R. Rivera, P. Kotzias, and J. Caballero. AVClass: A tool for massive malware labeling. In *Proc. Int. Symp. Research*

- in *Attacks, Intrusions, and Defenses (RAID)*, pages 230–253, 2016.
- [24] H. Shimodaira. Improving predictive inference under covariate shift by weighting the log-likelihood function. *Journal of statistical planning and inference*, 90(2):227–244, 2000.
 - [25] S. Sinha. State of IoT 2025: Number of connected IoT devices growing 14% to 21.1 billion globally. [Online]. Available: <https://iot-analytics.com/number-connected-iot-devices/>, 2024. Accessed: Feb. 2026.
 - [26] B. Sun and K. Saenko. Deep coral: Correlation alignment for deep domain adaptation. In *European conference on computer vision*, pages 443–450. Springer, 2016.
 - [27] D. Vasan, M. Alazab, S. Venkatraman, J. Akram, and Z. Qin. MTHAEL: Cross-architecture IoT malware detection based on neural network advanced ensemble learning. *IEEE Transactions on Computers*, 69(11):1654–1667, 2020.
 - [28] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio. Graph attention networks. *arXiv preprint arXiv:1710.10903*, 2017.
 - [29] S. VenkataKeerthy, S. Banerjee, S. Dey, Y. Andaluri, R. PS, S. Kalyanasundaram, F. M. Quintão Pereira, and R. Upadrasta. Vexir2vec: an architecture-neutral embedding framework for binary similarity. *ACM Transactions on Software Engineering and Methodology*, 34(8):1–54, 2025.
 - [30] VirusTotal. VirusTotal – free online virus, malware and url scanner. [Online]. Available: <https://www.virustotal.com>. Accessed: Mar. 2026.
 - [31] J. Wang et al. Can a deep learning model for one architecture be used for others? Retargeted-Architecture binary code analysis. In *Proc. 32nd USENIX Security Symp.*, pages 7339–7356, 2023.
 - [32] J. Wang, Q. Zeng, and L. Luo. Learning cross-architecture instruction embeddings for binary code analysis in low-resource architectures. In *Findings of the Association for Computational Linguistics: NAACL 2024*, pages 1320–1332, 2024.